

Resenha Crítica: Facade (Design Pattern)

Nome: Eduarda Vieira Gonçalves

Curso: Engenharia de Software

Título: Resenha Crítica – Facade Design Pattern

Referência: Refactoring Guru – Facade Design Pattern

O padrão de projeto Facade, também conhecido como Fachada, é um padrão estrutural cujo objetivo é fornecer uma interface simplificada para um subsistema complexo de classes, bibliotecas ou frameworks. A ideia central desse padrão é esconder a complexidade interna de um sistema e disponibilizar apenas as funcionalidades essenciais para o cliente, tornando o código mais simples de usar e manter. Esse padrão é amplamente utilizado em sistemas de software para reduzir o acoplamento entre o código cliente e os subsistemas internos.

O problema que o padrão Facade busca resolver ocorre quando uma aplicação precisa interagir com um conjunto grande de classes de uma biblioteca ou framework complexo. Nessa situação, o desenvolvedor precisa inicializar diversos objetos, controlar dependências e chamar métodos em uma ordem específica, o que torna o código difícil de entender e manter. Como consequência, a lógica de negócio acaba ficando fortemente acoplada aos detalhes de implementação dessas classes, dificultando futuras alterações no sistema.

Como solução para esse problema, o padrão Facade propõe a criação de uma classe intermediária que serve como uma fachada para o subsistema. Essa classe oferece uma interface simples e direciona as chamadas do cliente para as classes apropriadas dentro do subsistema. Dessa forma, o cliente não precisa conhecer a complexidade interna do sistema, apenas a interface fornecida pela fachada. Isso facilita a manutenção e permite que o subsistema seja modificado sem impactar diretamente o código cliente.

A estrutura do padrão Facade é composta basicamente por três elementos principais: o cliente, a fachada e o subsistema complexo. O cliente interage apenas com a fachada, a fachada coordena as chamadas e interage com o subsistema, e o subsistema contém as classes responsáveis pela execução das funcionalidades reais do sistema. Um ponto importante é que as classes do subsistema não têm conhecimento da existência da fachada, ou seja, elas continuam funcionando normalmente e podem se comunicar entre si diretamente.

Entre as principais vantagens do padrão Facade está a redução da complexidade do sistema e a diminuição do acoplamento entre as classes. Ao centralizar o acesso ao subsistema em uma única interface, o sistema torna-se mais organizado, mais fácil de entender e mais simples de manter. Além disso, se o subsistema for alterado ou substituído, normalmente apenas a classe fachada precisa ser modificada, sem necessidade de alterar o restante do sistema. Por outro lado, uma desvantagem desse padrão é que a fachada pode se tornar uma classe muito grande e complexa, conhecida como “God Object”, concentrando responsabilidades demais em um único lugar.

Do ponto de vista prático, o padrão Facade é muito útil quando trabalhamos com APIs externas, bibliotecas complexas ou sistemas com muitas camadas. Em vez de o código

cliente acessar diretamente várias classes diferentes, ele acessa apenas a fachada, que simplifica a comunicação e organiza as operações do sistema. Esse padrão também ajuda a estruturar sistemas em camadas, permitindo que diferentes partes do sistema se comuniquem apenas através das fachadas, reduzindo o acoplamento entre os módulos.

Refletindo sobre a aplicação desse padrão em projetos de desenvolvimento de software, é possível perceber que ele pode ser utilizado para organizar melhor sistemas que utilizam APIs externas ou múltiplos serviços. Por exemplo, em um sistema como o Duda Flix, ao invés de o sistema acessar diretamente várias funções relacionadas à API de filmes, busca de dados, favoritos e autenticação, seria possível criar uma classe fachada responsável por centralizar essas operações, como por exemplo um serviço chamado MovieService ou DudaFlixFacade. Dessa forma, o restante do sistema faria chamadas apenas para essa fachada, tornando o código mais organizado e reduzindo o acoplamento entre os componentes do sistema.

Dessa forma, o padrão Facade demonstra ser uma solução importante para reduzir a complexidade de sistemas e melhorar a organização do código. Embora não adicione novas funcionalidades ao sistema, ele melhora significativamente a estrutura e a manutenção do software, sendo especialmente útil em sistemas grandes ou que utilizam múltiplas bibliotecas e serviços externos. Assim, o Facade se apresenta como um padrão fundamental para a construção de sistemas mais organizados, desacoplados e de fácil manutenção.